

Projectdocumentatie – AquaSense (Waterview)

Slim zwembadmonitorsysteem



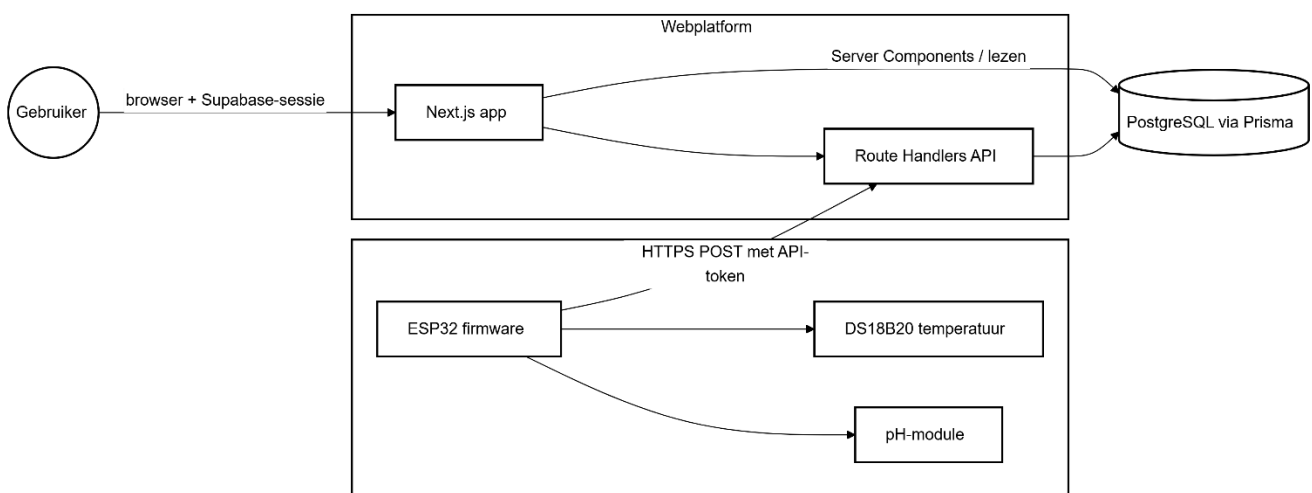
Sven Van Leemput
Integratieproject - Toegepaste Informatica
AquaSense

Systeemuitleg – AquaSense (Waterview)

Dit document legt uit hoe het AquaSense-systeem werkt. Je ziet hoe de verschillende onderdelen samenwerken: de firmware op de ESP32, de webapplicatie (backend en frontend in één Next.js-project), de databank en de gebruikersrollen. Het bouwt verder op het solution design en gaat dieper in op de werking, datastromen en technische keuzes.

1. Doel en globale werking

Het systeem meet zwembadwater (minimaal temperatuur en pH in het MVP), stuurt die gegevens periodiek naar een centrale API en slaat ze op in een relationele databank. Via het webdashboard kun je de actuele waarden en grafieken bekijken. Zo zie je snel hoe het water evolueert. Beheerders hebben extra rechten. Zij kunnen apparaten registreren, drempelwaarden instellen, logs bekijken en gebruikers beheren.



2. Componenten: taken en verantwoordelijkheden

2.1 IoT-device

Taken van de ESP32

De ESP32 leest op vaste momenten de sensoren uit:

- Watertemperatuur via de DS18B20 (OneWire)
- pH via een analoge ingang (GPIO34), met filtering en stabiliteitscontrole

De pH-waarde wordt berekend met kalibratiegegevens die op het toestel zelf worden opgeslagen (in flash via Preferences). Het systeem ondersteunt:

- Eénpuntskalibratie (pH 7)
- Tweepuntskalibratie (pH 7 + pH 4 of pH 7 + pH 10)
- Temperatuurcompensatie via de Nernst-helling

De ESP32 geeft ook lokale feedback:

- LEDs (groen, geel, rood)
- Buzzer
- Schakelen tussen demo-modus en live-modus via een drukknop

Voor testen en diagnose kun je optioneel seriële commando's gebruiken.

In live-modus, en als WiFi werkt, stuurt het toestel ongeveer elke 30 seconden een HTTPS POST naar de server. Dit gebeurt via een JSON-body (zie interval `WATERVIEW\_POST\_INTERVAL\_MS`).

Verantwoordelijkheid van de ESP32

De microcontroller is de bron van waarheid voor de metingen. Ruwe sensorwaarden en kalibratie blijven op het toestel zelf. De server ontvangt vooral de verwerkte meetwaarden. Die sluiten aan bij het datamodel en zijn klaar voor opslag en weergave.

Netwerklaag

De module "cloud_upload.h" regelt de communicatie met de server.

- Gebruikt een beveiligde TLS-verbinding (WiFiClientSecure)
- Stuurt data naar: `https://<INGEST\_HOST>/api/measurements`
- Voegt een header toe: `X-Device-Token` met de API-sleutel (uit secrets.h)

Belangrijk detail voor de databinding

De JSON-payload bevat een "deviceld" (bijvoorbeeld `waterview-esp32-01`). Maar de server gebruikt deze waarde niet om de meting te koppelen. De koppeling gebeurt alleen via de API-sleutel in de header. Dat heeft twee voordelen:

- Zonder geldige API-sleutel kan niemand metingen versturen
- Een fout in "deviceld" kan geen bestaande data overschrijven

2.2 Backend / API

Taken van de API

De backend verwerkt alle inkomende en uitgaande data via route handlers in Next.js (src/app/api/).

1. Ingest endpoint

- POST /api/measurements ontvangt meetgegevens van het device
- Valideert de data met Zod (measurementIngestSchema)
- Zoekt het juiste device via de token in de header
- Slaat de meting op in de Measurement-tabel

Bij een succesvolle aanvraag:

- Wordt "lastSeenAt" van het device bijgewerkt
- Wordt een auditlog aangemaakt

2. Beheer-API's

Onder /api/admin/...zitten alle beheerfuncties:

- Devices aanmaken en beheren (met een API-key die je maar één keer ziet)
- Drempelwaarden instellen (globaal of per device)
- Gebruikersprofielen beheren

Deze routes werken alleen voor ingelogde gebruikers met adminrechten. De controle gebeurt via de Supabase-sessie en "Profile.role === ADMIN".

3. Health check

- GET /api/health voert een eenvoudige database-check uit voor monitoring.

Verantwoordelijkheid van de API

Alle serverlogica die externe clients (browser of device) ziet, loopt door route handlers. Validatie gebeurt met Zod. Interne fouten (zoals stack traces) worden niet naar de client gestuurd

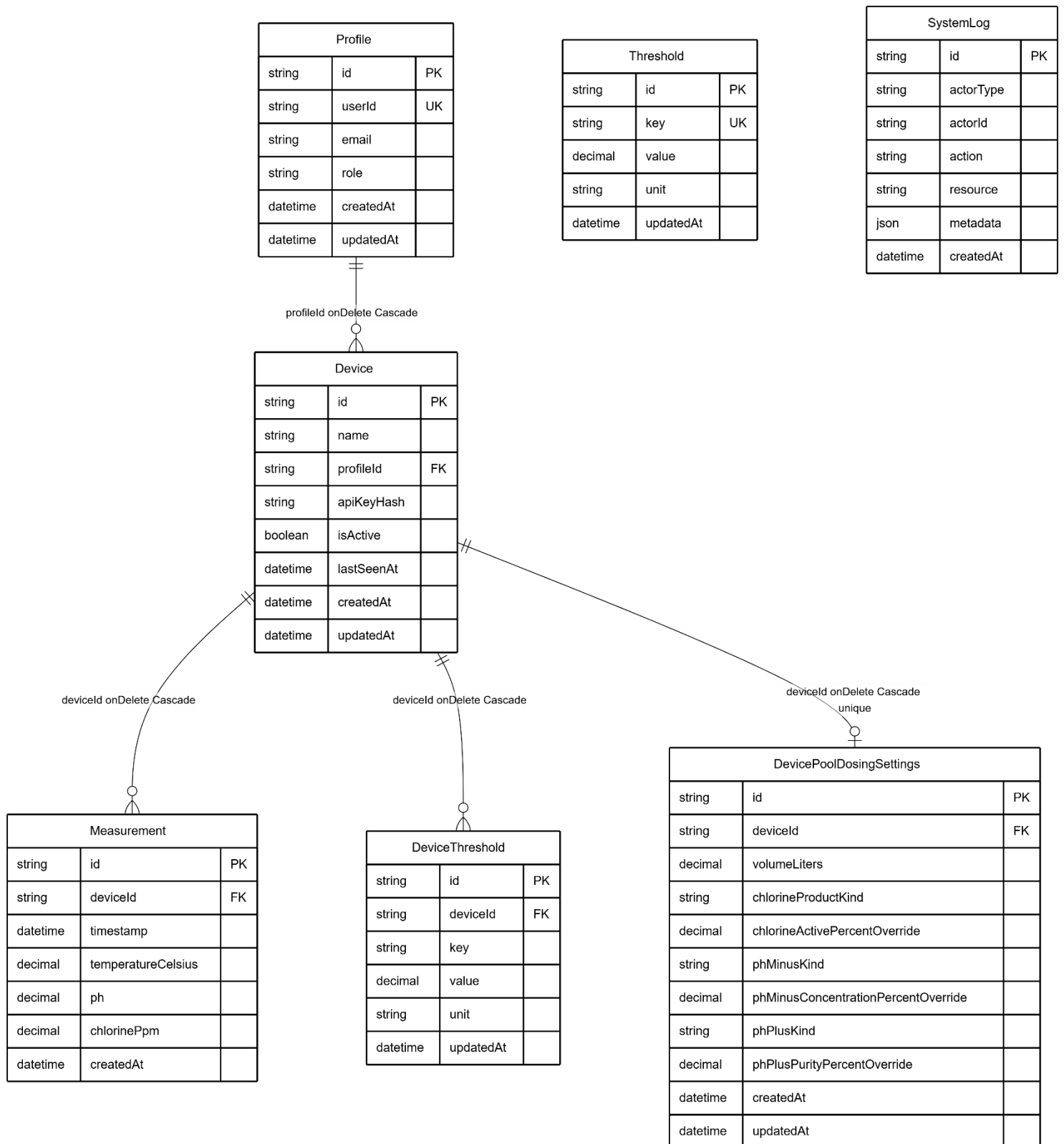
2.3 Databank (Prisma + PostgreSQL)

Het schema (prisma/schema.prisma) definieert onder andere:

Model	Rol
Profile	Koppeling tussen Supabase auth.users en applicatie: e-mail, rol (USER / ADMIN), eigenaar van devices.
Device	Logisch zwembadmonitor: bevat een hash van de device-API-key, isActive, lastSeenAt.
Measurement	Tijdreeks van temperatureCelsius, ph, chlorinePpm (optioneel voor MVP).
Threshold / DeviceThreshold	Platformdefaults en per device grenzen voor dashboards en statuskleuren.
DevicePoolDosingSettings	Volume en producttypes voor schattings-UI (dosering).
SystemLog	Audit- en gebeurtenislog (actorType, action, optioneel metadata JSON).

Verantwoordelijkheid

PostgreSQL is de centrale databank van het systeem. Hier worden alle gegevens opgeslagen. De server gebruikt Prisma om met de databank te werken. Dit gebeurt in de route handlers en Server Components. Prisma zorgt voor typeveilige toegang, waardoor fouten sneller opvallen en de code betrouwbaarder blijft.

Relationeel model

2.4 Webcomponent - dashboard en admin

Dashboard

- Layout dwingt login via Supabase: bij eerste bezoek wordt een Profile aangemaakt (ensureProfile), waarbij de eerste gebruiker in het systeem automatisch ADMIN krijgt en latere gebruikers standaard USER.
- Het hoofddashboard toont alleen de devices die bij jouw profiel horen (profileld).
- De meetgeschiedenis (/dashboard/measurements) gebruikt paginatie en een limiet (CHART_SAMPLE_LIMIT) voor grafieken, zodat de performance stabiel blijft.

Admin

- Aparte layout: alleen gebruikers met adminrol krijgen deze routes te zien: anderen worden naar /dashboard gestuurd.
- Functies zoals device-setup (wizard), drempels per device, platforminstellingen, logfilter en gebruikersbeheer hangen hier onder.

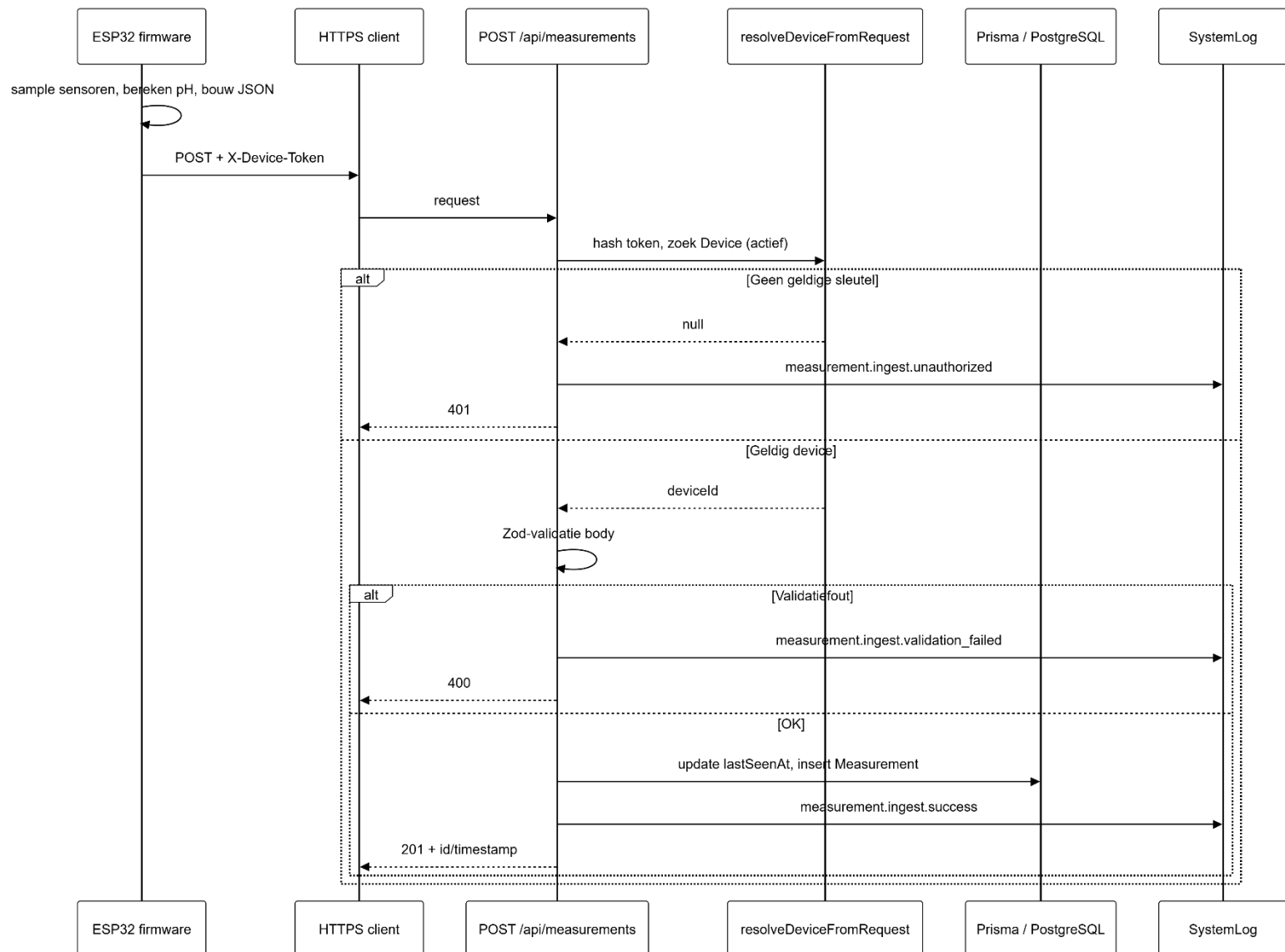
Verantwoordelijkheid

De UI is gebouwd in React, binnen hetzelfde Next.js-project als de API.

- Data wordt zoveel mogelijk op de server opgehaald (Server Components met Prisma)
- Interactieve acties (zoals formulieren) sturen requests naar API-routes

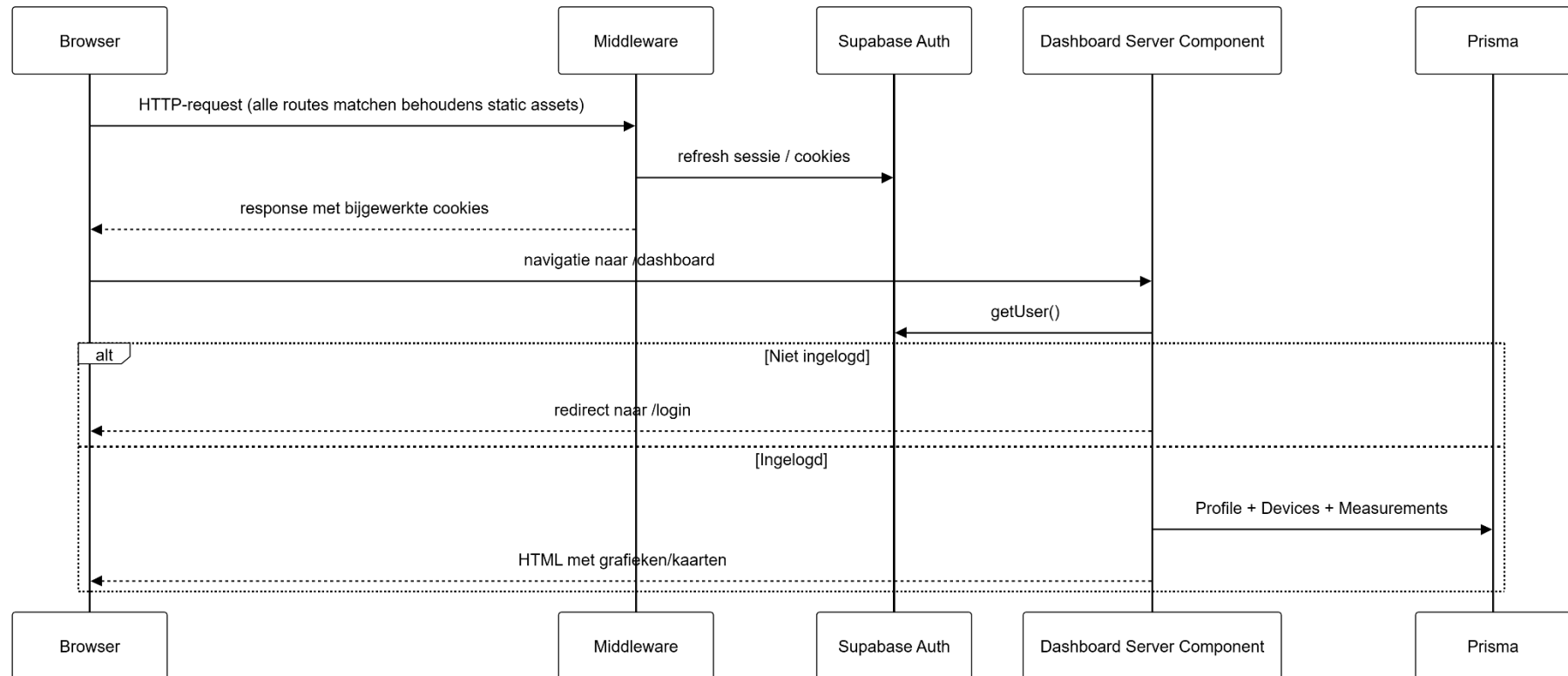
3. Datastromen en event flows

3.1 Meting van sensor tot databank

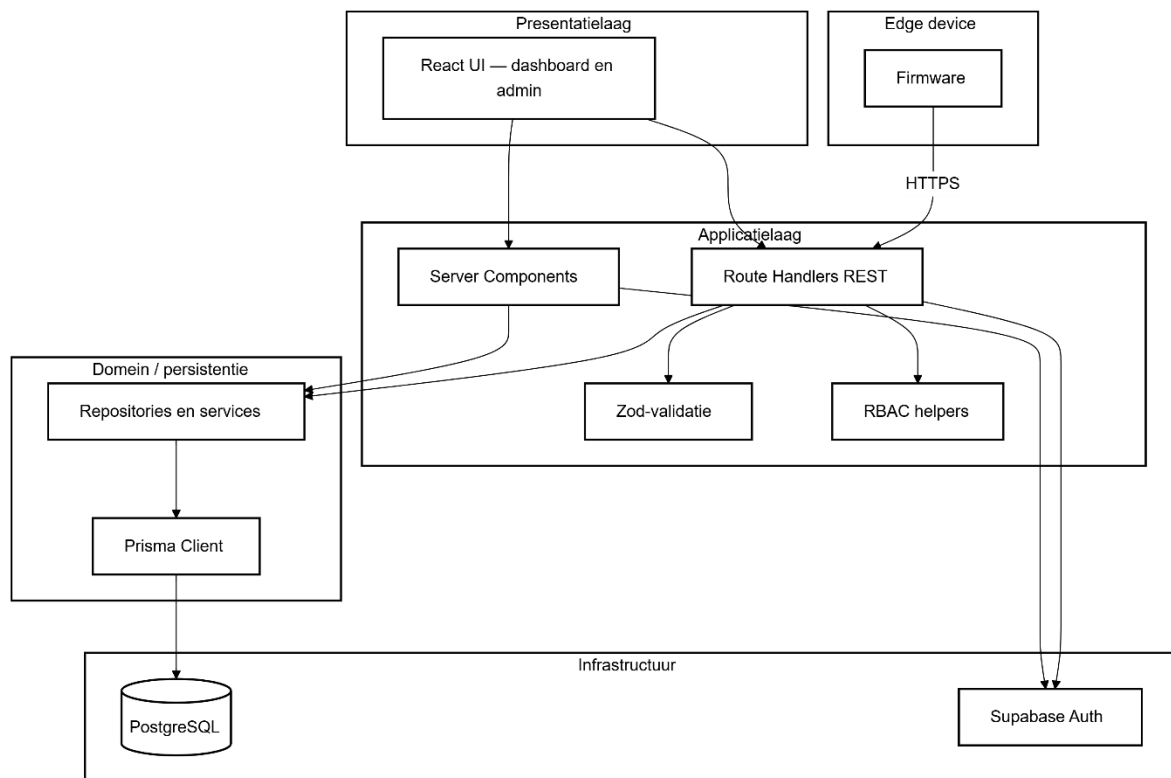


De firmware stuurt naast ph en optioneel temperatureCelsius veel extra velden mee voor diagnose; het Zod-schema op de server accepteert alleen de relevante meetvelden.

3.2 Gebruiker opent het dashboard



4. Architectuurdiagram (lagen)



5. Caching, sessies, data

- De Supabase-sessie wordt beheerd via “middleware.ts”. Voor bijna alle routes wordt “updateSession” uitgevoerd, zodat auth-cookies gesynchroniseerd zijn voor het renderen. Als de Supabase-config ontbreekt, slaat de middleware dit over. Dat is handig lokaal, maar in productie zijn deze variabelen nodig.
- Dashboardpagina’s gebruiken “export const dynamic = 'force-dynamic'”. Daardoor wordt caching van Next.js uitgeschakeld en krijg je telkens de meest recente data uit de databank. Dat is belangrijk voor een monitoringapp.
- Er is geen aparte cachelaag zoals Redis. Caching zit vooral in:
 - sessiecookies
 - browsercache van assets
 - beperkte queries (via “take”/”limit”) om zware databasecalls te vermijden

6. Rechten en rollen

Rol	Toegang
USER	Eigen dashboard, eigen devices en metingen, pool/dosering-instellingen waar voorzien in de UI.
ADMIN	Zelfde als user, plus admin-sectie: devices aanmaken (met API-key), thresholds beheren, logs, gebruikersrollen.

Admin-toegang wordt server-side afgedwongen, zowel in layouts als API-routes. Bij onvoldoende rechten geven admin-routes een 403 Forbidden.

7. Logging & audit

- De service createLog schrijft logs naar SystemLog. Elke log bevat o.a. een actorType (user, device, system), een action (zoals measurement.ingest.success) en optioneel extra metadata.
- De ingest-logica registreert fouten zoals mislukte authenticatie, validatieproblemen en interne errors. Gevoelige data wordt niet naar de client gestuurd.
- Beheeracties, zoals het aanmaken van een device, worden ook gelogd met de gebruiker als actor en een korte samenvatting.

Dit maakt controle mogelijk tijdens demo's en vormt een basis voor latere uitbreiding naar een uitgebreider logging systeem

8. Beveiliging

Tussen device en backend

- We gebruiken TLS voor een veilige verbinding (via WiFiClientSecure).
- Certificaat-pinning staat nog niet aan in het prototype. Dit is gemarkeerd als een TODO voor extra beveiliging.
- Het device stuurt een gedeeld secret mee via de header X-Device-Token.
- Alternatieven zoals X-API-Key of Authorization: Bearer werken ook.
- In de database slaan we alleen een SHA-256 hash van de key op, niet de echte sleutel.
- Je ziet de originele api-key maar één keer, bij het aanmaken van het device.

Tussen browser en backend

- Authenticatie verloopt via Supabase met een OAuth-flow.
- Na login kom je terug via /auth/callback.
- Controleren redirects extra met sanitizeInternalPath om misbruik te voorkomen.
- Admin-acties werken alleen als je bent ingelogd en de juiste rol hebt.

API-contract

- We valideren alle belangrijke input met Zod.
- Bij fouten (validatie of authenticatie) sturen we alleen korte en veilige foutmeldingen terug.

9. Koppeling met embedded stack en system stack (onderzoek)

De implementatie sluit aan bij de keuzes uit het onderzoek. Zowel de embedded kant als de system stack zijn bewust eenvoudig gehouden, zodat het project haalbaar blijft als MVP.

Embedded stack

- De ESP32 biedt genoeg rekenkracht en heeft ingebouwde WiFi.
- We gebruiken het Arduino ecosysteem met standaardbibliotheken zoals WiFi, HTTPClient, Preferences en sensorlibraries (OneWire, DallasTemperature).
- De DS18B20 meet temperatuur digitaal en is geschikt voor gebruik in water, zoals bij een zwembad.
- De pH-sensor werkt analoog.
 - We filteren de metingen in software.
 - We controleren of de waarde stabiel is voordat we ze gebruiken.
 - Kalibratiegegevens slaan we op.
 - Dit past bij de beperkingen van goedkopere sensoren die minder nauwkeurig zijn.

System / developer stack

- We gebruiken een Next.js-app.
 - UI en API zitten in dezelfde codebase.
 - Dit maakt deployen eenvoudiger, zeker voor een MVP op platforms zoals Vercel.
- PostgreSQL met Prisma zorgt voor een duidelijk relationeel model.
 - Dit past goed bij data zoals devices, metingen en drempelwaarden.
 - Een uitbreiding naar bijvoorbeeld TimescaleDB is mogelijk, maar valt buiten de scope.
- Authenticatie loopt via Supabase.
 - Dit vermijdt dat we zelf login- en wachtwoordbeheer moeten bouwen.
 - Sessies en tokens worden automatisch afgehandeld.

API-model

- We gebruiken een REST API voor:
 - data van devices (metingen)
 - admin-acties
- Er is geen realtime communicatie (zoals WebSockets).
 - Devices sturen periodiek data door, en dat is voldoende voor deze fase.
 - Realtime updates kunnen later toegevoegd worden als uitbreiding.